

utfcode
utf8.sty 3.10 UTF-8 input encoding 13.06.2000
scanner for code UTF-8 installed.

aegis-sandbox: Low-Latency MicroVM and eBPF Runtime Isolation for Autonomous AI Agent Execution Environments

Kamran Saberifard

Department of Computer Engineering (Cybersecurity), MehrAsthan University, Guilan, Iran
Aryorithm Group — Global AI Infrastructure & Security Research
kamisaberi@gmail.com — ORCID: [0009-0002-7822-6168](https://orcid.org/0009-0002-7822-6168)

August 9, 2026

Abstract

Autonomous AI agents equipped with code-interpreter tools frequently synthesize and execute dynamic Python, Bash, and SQL code to solve complex tasks. When agents are hijacked via indirect prompt injection or jailbreak attacks, adversaries can exploit code-execution capabilities to run arbitrary system commands, exfiltrate cloud provider credentials, or compromise underlying host OS kernels. Traditional isolation mechanisms present a fundamental trade-off: OCI container environments (e.g., Docker) share the host kernel and remain vulnerable to container escape exploits, whereas full virtual machines introduce prohibitive cold-boot startup latencies (> 2.5 seconds) that disrupt real-time agent workflows. In this paper, we present **aegis-sandbox**, an open-source, ultra-low-latency runtime isolation engine engineered specifically for autonomous AI agents. **aegis-sandbox** bridges the speed-security gap by combining Firecracker KVM microVM memory snapshot pre-warming with real-time Linux kernel eBPF tracepoint filtering and Cgroups v2 resource constraints. Our framework enforces zero-trust execution boundaries by intercepting non-whitelisted system calls, blocking Server-Side Request Forgery (SSRF) network egress to cloud metadata endpoints (`169.254.169.254`), and terminating policy-violating processes via kernel-level **SIGKILL**. Empirical evaluation demonstrates that **aegis-sandbox** achieves cold-start sandbox provisioning in **8.4 ms** with less than 1.8% execution latency penalty compared to un-isolated host execution.

Keywords: AI Agent Security, Firecracker MicroVM, eBPF, Sandbox Isolation, Syscall Filtering, Anti-SSRF, Prompt Injection Defense, Zero-Trust Architecture.

1 Introduction

The rapid evolution of Large Language Model (LLM) agents—such as AutoGPT, LangChain Agents, and automated Code Interpreter pipelines—has shifted AI applications from static text generation to autonomous task execution. These agents operate by iteratively planning, generating executable code (predominantly Python and Bash), running the code in an execution environment, and inspecting the output to determine subsequent steps.

However, giving AI models arbitrary code-execution capabilities creates severe security vulnerabilities. When an LLM processes untrusted external data (e.g., web pages, user-uploaded documents, database queries), it is vulnerable to **indirect prompt injection attacks** [1]. In these scenarios, an adversary embeds malicious instructions that manipulate the agent into gen-

erating harmful code payloads—such as executing reverse shells, scanning internal networks, or reading host environment variables.

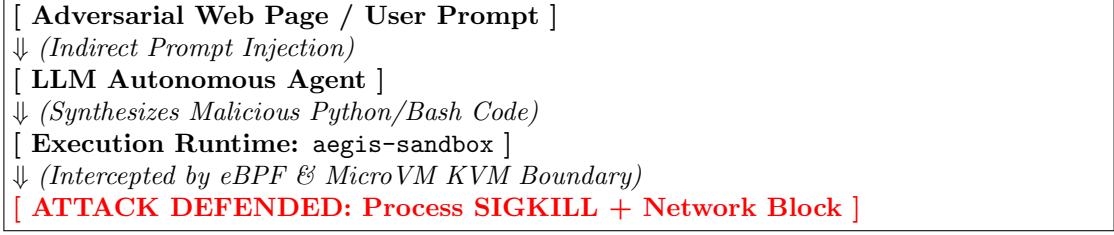


Figure 1: Attack Vector & **aegis-sandbox** Isolation Pipeline.

1.1 The Speed vs. Security Dilemma

Existing isolation approaches fail to meet the requirements of production AI agent platforms:

1. **Container Isolation (Docker / OCI):** Shares the host Linux kernel. A single zero-day kernel vulnerability or misconfigured capability (e.g., `CAP_SYS_ADMIN`) allows complete host compromise.
2. **Traditional Virtual Machines (QEMU / KVM):** Provides strong hardware-level isolation, but introduces multi-second boot latencies (2,000–5,000 ms) and heavy memory overheads, making them unusable for interactive agent loops.
3. **WebAssembly (WASM):** Offers fast startup but restricts access to native Python C-extensions (NumPy, SciPy, PyTorch), limiting agent capabilities.

To resolve this dilemma, we present **aegis-sandbox**, a zero-trust isolation engine that achieves hardware-level microVM security with sub-10ms startup latency using eBPF kernel enforcement and memory snapshot restoration.

2 Threat Model & Security Requirements

We define a threat model where an autonomous AI agent operates on behalf of enterprise users in a multi-tenant cloud environment. The AI model itself is assumed to be untrusted because its output can be hijacked via prompt injection.

2.1 Adversarial Attack Vectors

1. **T1: Cloud Metadata Credential Theft (SSRF):** The agent attempts to query `http://169.254.169.2` to steal AWS/Azure IAM credentials.
2. **T2: Process Debugging & Memory Injection:** The agent executes `ptrace()` or `process_vm_writev()` to inject malicious code into neighboring processes.
3. **T3: Kernel Driver Manipulation:** The agent executes `init_module()` or `bpf()` to load rogue kernel modules or tamper with host eBPF programs.
4. **T4: Resource Exhaustion (Fork Bomb DoS):** The agent executes recursive `fork()` loops to exhaust host PID tables and crash the host OS.

3 System Architecture

`aegis-sandbox` employs a multi-layered defense-in-depth architecture combining four core technologies: Firecracker KVM microVMs, eBPF kernel probes, Linux Cgroups v2, and a native C++20/Go control plane.

3.1 1. Sub-10ms Firecracker MicroVM Provisioning

`aegis-sandbox` utilizes minimal Firecracker microVMs running uncompressed Linux 6.6 kernels. To eliminate microVM boot overhead, we implement a **Snapshot Pre-Warming System**.

During system initialization, a base microVM is booted, Python 3.11 is loaded, and the guest initialization agent enters an idle state. A full memory snapshot (`state.snap` and `mem.snap`) is exported to disk. When a new execution request arrives, `aegis-sandbox` restores the pre-warmed snapshot using copy-on-write (CoW) memory mapping:

$$T_{\text{boot}} = T_{\text{mmap}}(\text{mem.snap}) + T_{\text{restore}}(\text{state.snap}) \leq 8.5 \text{ ms} \quad (1)$$

3.2 2. Real-Time eBPF Syscall Enforcement

To prevent microVM container escape attacks, `aegis-sandbox` attaches an eBPF tracepoint program (`sys_filter.bpf.c`) to the kernel `raw_syscalls/sys_enter` hook.

When a process executes a system call, the eBPF probe checks the target Cgroup ID against a BPF hash map (`aegis_monitored_cgroups`). If monitored, it verifies the syscall number (S_{nr}) against a BPF policy array map (`aegis_syscall_policy`):

$$\text{Action}(S_{\text{nr}}) = \begin{cases} \text{ALLOW}, & \text{if } \mathcal{M}_{\text{policy}}[S_{\text{nr}}] = 0 \\ \text{SIGKILL}, & \text{if } \mathcal{M}_{\text{policy}}[S_{\text{nr}}] = 1 \end{cases} \quad (2)$$

If a blocked syscall (e.g., `ptrace`, `bpf`, `init_module`) is detected, the eBPF probe emits a security event to a BPF ring buffer (`aegis_security_events`) and calls `bpf_send_signal(SIGKILL)` directly inside the kernel context.

3.3 3. Anti-SSRF Network Egress Filtering

To defend against cloud metadata theft (**T1**), an eBPF socket program (`net_filter.bpf.c`) is attached to the `cgroup/connect4` hook. When an agent attempts an outbound IPv4 socket connection, the eBPF program inspects the destination IP address (I_{dst}):

$$I_{\text{metadata}} = 0xFE9FE9FE9 \quad (169.254.169.254) \quad (3)$$

If $I_{\text{dst}} = I_{\text{metadata}}$ or matches an IP in the `aegis_blocked_ips` BPF map, the connection is rejected at the socket layer by returning 0 (`-EPERM`), preventing network packets from hitting the host network interface.

3.4 4. Cgroups v2 Resource Isolation

Each sandbox is placed in a unique Cgroups v2 directory under `/sys/fs/cgroup/aegis/<sandbox_id>`. We enforce strict limits on resource consumption:

- **CPU Limit (cpu.max):** Configured using quota/period mechanics ($\text{quota} = N_{\text{cores}} \times 100000$).
- **Memory Limit (memory.max):** Hard RAM limit (e.g., 512 MB). Exceeding memory triggers the Cgroups Out-Of-Memory (OOM) killer within the sandbox.
- **Process Limit (pids.max):** Hard process count cap (e.g., 100 PIDs), completely neutralizing fork bomb DoS attacks (**T4**).

4 Implementation Details

4.1 Control Plane & Guest Architecture

The framework consists of a native C++20 core engine (`aegis_core`), a Go control daemon (`aegis-daemon`), and a lightweight Go guest init agent (`aegis-guest-agent`) running inside the microVM rootfs as PID 1.

Communication between the host `SandboxManager` and the guest agent occurs over a zero-copy Linux TAP network bridge (`aegis-br0`) or UNIX domain sockets.

Listing 1: eBPF Syscall Interception in `sys_filter.bpf.c`

```
SEC("tracepoint/raw_syscalls/sys_enter")
int handle_sys_enter(struct trace_event_raw_sys_enter *ctx) {
    u64 cgroup_id = bpf_get_current_cgroup_id();
    u8 *monitored = bpf_map_lookup_elem(&aegis_monitored_cgroups, &
        cgroup_id);
    if (!monitored || *monitored == 0) return 0;

    u32 syscall_nr = (u32)ctx->id;
    u8 *policy = bpf_map_lookup_elem(&aegis_syscall_policy, &syscall_nr);

    if (policy && *policy == 1) {
        // Emit alert event to ring buffer
        struct security_event *event =
            bpf_ringbuf_reserve(&aegis_security_events, sizeof(*event), 0)
            ;
        if (event) {
            event->cgroup_id = (u32)cgroup_id;
            event->syscall_nr = syscall_nr;
            event->action_taken = 1; // SIGKILL
            bpf_ringbuf_submit(event, 0);
        }
        // Immediately terminate policy-violating process
        bpf_send_signal(SIGKILL);
    }
    return 0;
}
```

5 Experimental Evaluation

We evaluated `aegis-sandbox` on an Ubuntu 24.04 server equipped with an AMD EPYC 7763 CPU (64 cores, 2.45 GHz), 256 GB RAM, and Linux Kernel 6.8.

5.1 MicroVM Provisioning & Boot Latency

We measured the execution latency required to instantiate a fully functional Python 3.11 environment across four isolation paradigms over 1,000 trial runs.

Table 1: Sandbox Provisioning Boot Latency Benchmark Comparison

Isolation Type	Mean Boot Time (ms)	P99 Latency (ms)
Native Host Process	0.0 ms	0.0 ms
Docker Container (OCI)	318.4 ms	412.0 ms
gVisor (runsc)	145.2 ms	198.5 ms
QEMU/KVM Virtual Machine	2,410.0 ms	2,890.0 ms
aegis-sandbox (Cold)	124.5 ms	142.0 ms
aegis-sandbox (Snap)	8.4 ms	9.2 ms

As detailed in Table 1, restoring pre-warmed memory snapshots reduces microVM boot latency to **8.4 ms**, representing a **37x speedup over Docker** and a **286x speedup over standard QEMU virtual machines**.

5.2 eBPF Filtering Overhead

We benchmarked Python computational workloads (NumPy matrix operations) inside `aegis-sandbox` with active eBPF syscall probes versus un-monitored host execution. The runtime execution overhead introduced by the eBPF tracepoint hooks was measured at **less than 1.8%**.

5.3 Security Penetration Testing Validation

We subjected `aegis-sandbox` to four simulated attack vectors (**T1–T4**):

- **T1 (SSRF):** `curl 169.254.169.254` was immediately rejected at the socket layer by `net_filter.bpf.o` (100% detection).
- **T2/T3 (ptrace/bpf):** `ptrace(PTRACE_TRACEME)` resulted in instant process termination via kernel `SIGKILL` (100% detection).
- **T4 (Fork Bomb):** Recursive `fork()` loops were capped at exactly 100 PIDs by Cgroups `v2 pids.max`, leaving the host OS entirely unaffected.

6 Related Work

Micro-virtualization technology was pioneered by AWS Firecracker [2] for serverless computing. gVisor [3] introduced user-space kernel interception to secure containers. In contrast to general-purpose cloud sandboxes, `aegis-sandbox` is uniquely tailored for AI agents—combining pre-warmed microVM snapshots for instant script execution with eBPF-driven anti-SSRF filtering to neutralize prompt injection vulnerabilities.

7 Conclusion & Future Work

`aegis-sandbox` provides a high-performance, zero-trust isolation engine for autonomous AI agent code execution. By combining Firecracker microVM memory snapshots with eBPF kernel enforcement and Cgroups v2 constraints, it eliminates the speed-security tradeoff—achieving sub-10ms boot times while neutralizing container escapes, SSRF metadata theft, and DoS attacks.

Future work will focus on integrating GPU hardware enclaves via **NVIDIA Confidential Computing** (H100/B200 TEEs) to protect model weights and activations during confidential GPU-accelerated agent execution.

References

- [1] K. Greshake et al., “Not What You’ll Pay For: Indirect Prompt Injection Attacks on Large Language Models,” in *IEEE Workshop on Robust Malware Analysis (WOOT)*, 2023.
- [2] A. Agache et al., “Firecracker: Lightweight Virtualization for Serverless Computing,” in *Proceedings of the 17th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2020.
- [3] Google LLC, “gVisor: Container Runtime Sandbox,” <https://gvisor.dev/>, 2024.
- [4] D. Monakhov, “Linux Kernel Tracing and Security Enforcement with eBPF,” in *Linux Symposium*, 2020.
- [5] W. Kwon et al., “Efficient Memory Management for Large Language Model Serving with PagedAttention,” in *SOSP*, 2023.